

Week 7 Assignment Report

Incremental Data Processing & Slowly Changing Dimension (SCD)Using MERGE

Objective

The objective of this assignment was to understand how Delta Lake manages incremental data processing. The workflow involved loading customer data into a Delta table, performing data cleaning, simulating incremental data, applying the Delta Lake MERGE operation to update existing records and insert new ones, validating the results, and displaying the final dataset.

Step 1: Loading the Customer Dataset

The initial customer dataset (customer_master.csv) was uploaded to Databricks and loaded into a Spark DataFrame using PySpark. Schema inference and header recognition were enabled to correctly identify the data types and column names.

06:08 PM (19s)

1

```
# STEP 1: Load dataset into Delta Table
# READ CSV
df_master_raw = (
    spark.read
      .option("header", True)
      .option("inferSchema", True)
      .csv("/Volumes/workspace/default/dataset/customer_master.csv")
)

display(df_master_raw)
```

> [See performance \(1\)](#)

> df_master_raw: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, name: string ... 4 more fields]

Table

	customer_id	name	email	city	status	updated_at
1	1	Ravi Sharma	ravi.sharma@example.com	Jaipur	active	2026-01-01
2	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
3	3	Suresh Kumar	suresh.kumar@example.com	Mumbai	active	2026-01-01
4	4	Priya Singh	priya.singh@example.com	Bangalore	active	2026-01-01
5	5	Amit Joshi	amit.joshi@example.com	Pune	active	2026-01-01
6	6	Neha Gupta	neha.gupta@example.com	Delhi	active	2026-01-01
7	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
8	8	Kiran Rao	kiran.rao@example.com	Hyderabad	active	2026-01-01
9	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
10	10	Meena Iyer	meena.iyer@example.com	Coimbatore	active	2026-01-01
11	11	Vikram Malhotra	vikram.malhotra@example.co...	Lucknow	active	2026-01-01
12	12	Pooja Reddy	null	Vizag	active	2026-01-01
13	13	Arjun Nair	arjun.nair@example.com	Kochi	inactive	2026-01-01
14	14	Sneha Kapoor	sneha.kapoor@example.com	Jaipur	active	2026-01-01
15	15	Rohit Desai	rohit.desai@example.com	Surat	active	2026-01-01

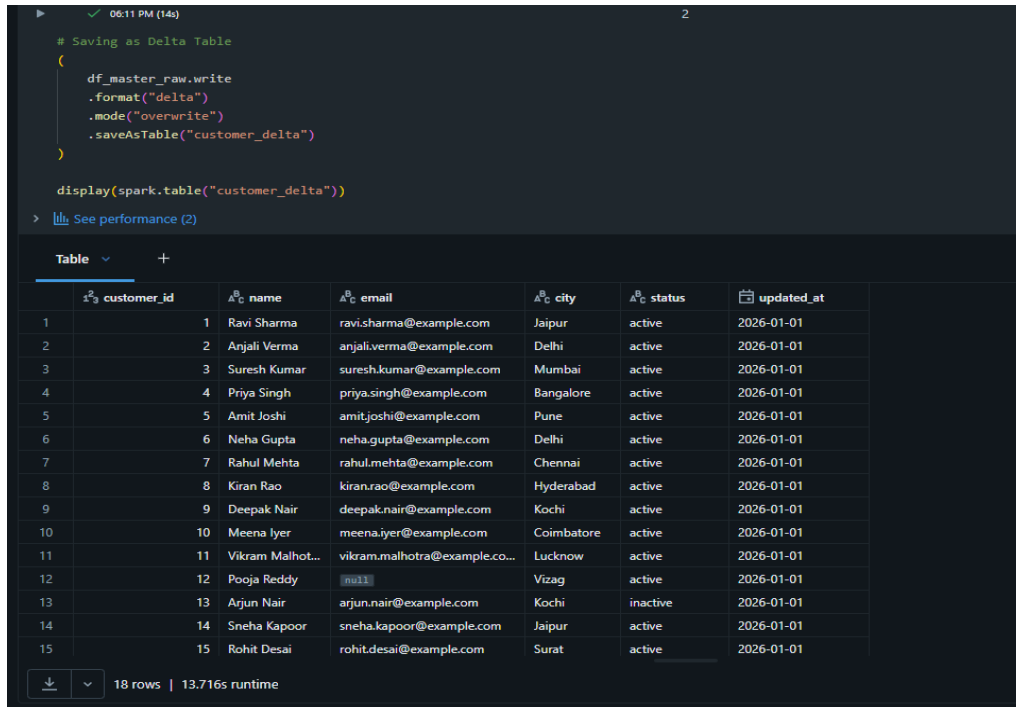
18 rows | 18.843s runtime

Insight:

- Successfully loaded the master dataset.
- Verified the schema and column names.
- Confirmed that the dataset was ready for preprocessing.

Step 2: Creating the Delta Table

The DataFrame was stored in Delta format. Delta Lake provides ACID transactions, schema enforcement, versioning, and efficient support for updates and merges.



The screenshot shows a Jupyter Notebook interface. The top part contains a code cell with the following Python code:

```
# Saving as Delta Table
(
    df_master_raw.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("customer_delta")
)

display(spark.table("customer_delta"))
```

Below the code cell, there is a link that says "See performance (2)". The bottom part of the screenshot shows a table preview with the following columns: customer_id, name, email, city, status, and updated_at. The table contains 15 rows of data.

	customer_id	name	email	city	status	updated_at
1	1	Ravi Sharma	ravi.sharma@example.com	Jaipur	active	2026-01-01
2	2	Anjali Verma	anjali.verma@example.com	Delhi	active	2026-01-01
3	3	Suresh Kumar	suresh.kumar@example.com	Mumbai	active	2026-01-01
4	4	Priya Singh	priya.singh@example.com	Bangalore	active	2026-01-01
5	5	Amit Joshi	amit.joshi@example.com	Pune	active	2026-01-01
6	6	Neha Gupta	neha.gupta@example.com	Delhi	active	2026-01-01
7	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
8	8	Kiran Rao	kiran.rao@example.com	Hyderabad	active	2026-01-01
9	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
10	10	Meena Iyer	meena.iyer@example.com	Coimbatore	active	2026-01-01
11	11	Vikram Malhotra	vikram.malhotra@example.co...	Lucknow	active	2026-01-01
12	12	Pooja Reddy	[null]	Vizag	active	2026-01-01
13	13	Arjun Nair	arjun.nair@example.com	Kochi	inactive	2026-01-01
14	14	Sneha Kapoor	sneha.kapoor@example.com	Jaipur	active	2026-01-01
15	15	Rohit Desai	rohit.desai@example.com	Surat	active	2026-01-01

At the bottom of the table preview, there is a download icon, a dropdown menu, and the text "18 rows | 13.716s runtime".

Insight:

- Successfully converted the dataset into a Delta table.
- Enabled efficient incremental processing using Delta Lake features.

Step 3: Data Cleaning

The loaded dataset was cleaned before storing it in Delta format. Null values were handled appropriately, and duplicate records were removed to ensure data consistency and quality.

Cleaning operations included:

- Handling missing (null) values.
- Removing duplicate records.
- Verifying the cleaned dataset.


```

%sql
MERGE INTO customer_delta AS target
USING incremental_view AS source
ON target.customer_id = source.customer_id

WHEN MATCHED THEN
UPDATE SET
    target.name = source.name,
    target.email = source.email,
    target.city = source.city,
    target.status = source.status,
    target.updated_at = source.updated_at

WHEN NOT MATCHED THEN
INSERT (customer_id, name, email, city, status, updated_at)
VALUES (source.customer_id, source.name, source.email, source.city, source.status, source.updated_at)

```

> [See performance \(1\)](#)

	num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
1	8	5	0	3

1 row | 9.282s runtime

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Insight:

- Existing customer information was updated successfully.
- New customer records were inserted automatically.
- No manual update or insert operations were required.

Step 5: Validation

After completing the merge, the final Delta table was validated by checking:

- Total number of records.
- Duplicate records.
- Overall data consistency.

Validation confirmed that the merge operation completed successfully without introducing duplicate entries.

```

# STEP 5: Validation

print("Total row count after merge:", spark.table("customer_delta").count())

# check for duplicate customer_ids
duplicates = spark.table("customer_delta").groupBy("customer_id").count().filter("count > 1")
display(duplicates)
print("Duplicate customer_ids found:", duplicates.count())

```

> [See performance \(3\)](#)

duplicates: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, count: long]

Total row count after merge: 17

customer_id	count
-------------	-------

No rows returned

0 rows | 2.027s runtime

Duplicate customer_ids found: 0

Insight:

- Row count matched the expected output.
- No duplicate customer IDs were found.
- Data integrity was maintained after merging.

Step 6: Final Output

The final Delta table contained:

- Updated customer records.
- Newly inserted customer records.
- Clean and validated data.

The dataset accurately reflected the latest customer information after incremental processing.

06:25 PM (1s)9

STEP 6: Final Output

display(spark.table("customer_delta").orderBy("customer_id"))

> See performance (1)

Table +

	customer_id	name	email	city	status	updated_at
3	3	Suresh Kumar	suresh.kumar@example.com	Mumbai	active	2026-01-01
4	4	Priya Singh	priya.singh@example.com	Bangalore	active	2026-01-01
5	5	Amit Joshi	amit.joshi@example.com	Pune	inactive	2026-02-15
6	6	Neha Gupta	neha.gupta@example.com	Delhi	active	2026-01-01
7	7	Rahul Mehta	rahul.mehta@example.com	Chennai	active	2026-01-01
8	8	Kiran Rao	kiran.rao@example.com	Hyderabad	inactive	2026-02-15
9	9	Deepak Nair	deepak.nair@example.com	Kochi	active	2026-01-01
10	10	Meena Iyer	meena.iyer@example.com	Coimbatore	active	2026-01-01
11	11	Vikram Malhotra	vikram.malhotra@example.co...	Lucknow	active	2026-01-01
12	13	Arjun Nair	arjun.nair@example.com	Kochi	active	2026-02-15
13	14	Sneha Kapoor	sneha.kapoor@example.com	Mumbai	active	2026-02-15
14	15	Rohit Desai	rohit.desai@example.com	Surat	active	2026-01-01
15	17	Ritu Bhatia	ritu.bhatia@example.com	Chandigarh	active	2026-02-15
16	18	Manoj Tiwari	manoj.tiwari@example.com	Bhopal	active	2026-02-15
17	19	Farah Khan	farah.khan@example.com	Nagpur	active	2026-02-15

17 rows | 1.071s runtime

Insight:

- Successfully completed incremental data processing.
- Final dataset contained the latest customer information.
- Demonstrated the practical use of Delta Lake for real-world ETL workflows.

Key Learnings

During this assignment, the following concepts were learned:

- Loading CSV data into Apache Spark.
- Reading and writing Delta tables.
- Performing data cleaning using PySpark.

- Removing duplicate records.
- Handling null values.
- Understanding Delta Lake storage format.
- Performing MERGE operations for incremental data.
- Updating existing records and inserting new records.
- Validating processed data.
- Understanding SCD Type 1 implementation using Delta Lake.

Conclusion

This assignment demonstrated how Delta Lake simplifies incremental data processing in modern data engineering workflows. By combining Apache Spark with Delta Lake, it becomes possible to efficiently manage updates, inserts, and data validation while maintaining data consistency through ACID transactions.

The implementation successfully loaded customer data, cleaned the dataset, created a Delta table, processed incremental records using the MERGE operation, validated the results, and produced an updated final dataset. This workflow reflects a common real-world ETL process used in data engineering pipelines.